Architecture Overview

This document describes the high-level architecture of the Kiosk Sign-In System, how the components fit together, and the design decisions behind them.

System Components

The system is made up of two main applications that run simultaneously:

Kiosk Application -- A fullscreen Tkinter desktop GUI designed for a Windows touchscreen kiosk PC. This is the primary interface used by caregivers and staff on-site.

Web Dashboard -- A Flask web application that runs on the local network, providing administrators with analytics, data management, and export functionality from any browser on the same network.

Both applications share the same SQLite database file (`signin_data.db`), located inside the `kiosk_signin/` directory during development.

How They Run Together

The entry point is `kiosk_signin/app.py`. When launched, it starts the Flask dashboard in a background daemon thread on port 5000, then opens the Tkinter kiosk GUI on the main thread. This means a single process hosts both interfaces.

```
main thread:     Tkinter mainloop (kiosk GUI)
daemon thread:   Flask dev server (web dashboard on port 5000)
```

The Flask server runs with `debug=False` and `use_reloader=False` to avoid conflicts with Tkinter's main loop. Because the thread is a daemon, it terminates automatically when the kiosk window is closed.

The dashboard can also be run independently with `python dashboard/app.py` for situations where the kiosk is not needed.

Application Layer Diagram

```
+-------------------------------------------------+
|                    app.py                       |
|  (entry point, starts both kiosk + dashboard)   |
+-----+-------------------------------------------+
      |
      +-------> Tkinter Kiosk (main thread)
      |             |
      |             +-- screens/        (UI screens)
      |             +-- components/     (reusable widgets)
      |             +-- theme.py        (colours, fonts)
      |
      +-------> Flask Dashboard (daemon thread)
                    |
                    +-- dashboard/app.py      (routes + logic)
                    +-- dashboard/templates/  (Jinja2 HTML)
                    +-- dashboard/static/     (CSS)
      |
      +-------> Shared modules
                    |
                    +-- database.py       (SQLite data access)
                    +-- export_util.py    (Excel generation)
                    +-- email_notifier.py (SMTP notifications)
```

Screen Navigation (Kiosk)

The kiosk uses a screen stack pattern. All screens are instantiated once at startup and stacked on top of each other using Tkinter's `place` geometry manager. Navigation is handled by lifting the target screen to the top with `screen.lift()` and calling its `on_show()` method to refresh data.

```
TapScreen (idle)
   |
   v
OptionsScreen (main menu)
   |
   +-- SignInScreen --> SignatureScreen --> [success popup]
   +-- RunSheetsScreen --> SignatureScreen --> [success popup]
   +-- SocialScreen
   +-- AdminPasswordScreen --> AdminPanelScreen
                                    |
                                    +-- ProfileScreen (member edit)
                                    +-- StaffProfileScreen (staff edit)
```

The `App` class holds a dictionary of all screen instances and provides the `show_screen(name, **kwargs)` method that every screen uses for navigation. This centralised navigation avoids tight coupling between screens.

Database Access Pattern

All database access goes through `database.py`, which provides a set of plain functions (not a class). Each function opens its own connection using `get_conn()`, performs the query inside a `with` statement for automatic commit/rollback, and returns plain Python dictionaries.

This approach was chosen over an ORM for simplicity, given the system only has a single database file and a small number of tables. SQLite's WAL (Write-Ahead Logging) mode is enabled to allow the kiosk and dashboard to read/write concurrently without blocking each other.

Platform Handling

The kiosk adapts its behaviour based on the platform:

- **Windows** (the kiosk's target platform): Opens maximised, enables the built-in virtual keyboard (`VirtualKeyboard`) that appears when any `Entry` widget receives focus.
- **Linux** (used for development): Opens at a fixed 1280x720 resolution. The virtual keyboard is disabled; a system on-screen keyboard (e.g. Squeekboard, Onboard) would need to be handled by the window manager instead.

Build and Distribution

The project uses PyInstaller to create standalone executables. A GitHub Actions workflow builds binaries on release for two targets: Windows x64 (the kiosk's deployment platform) and Linux x64 (for development/testing). Each binary bundles the dashboard templates, static files, and all Python dependencies into a single file.

When running from a PyInstaller bundle, the database is stored in a writable location (`%LOCALAPPDATA%\Kiosk Sign-In\` on Windows) rather than alongside the executable, since the bundle's internal filesystem is read-only.